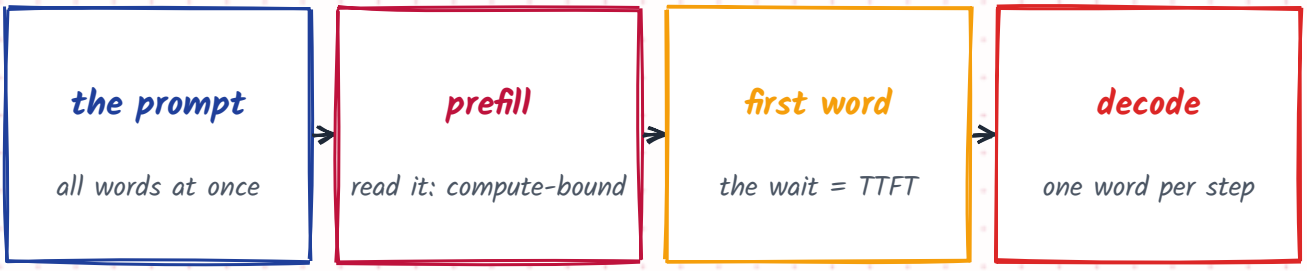
Inference and Serving

Lesson 6 of 12 · How to make a trained model answer fast, and cheaply.

In this lesson → ① Prefill and decode ② The KV cache, measured ③ Cache size ④ Continuous batching ⑤ Quantisation ⑥ Speculative decoding ⑦ Interview Q&A
Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 Serving: two phases



start fast = prefill · talk fast = decode

Phase	What happens	Limited by
prefill	the whole prompt is processed at once; builds the KV cache; gives the first token	compute
decode	one token per step; every step re-reads weights + cache	**memory bandwidth**

Remember
Start fast = prefill. Talk fast = decode. They need different fixes.

2 The clocks



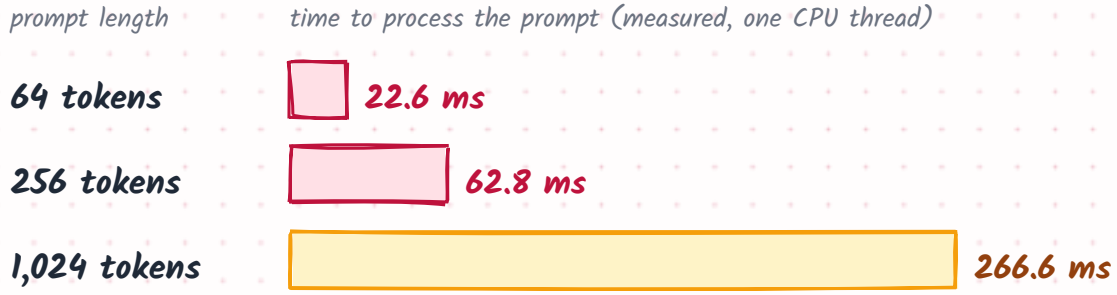
size your fleet by goodput, not raw tokens per second

total ≈ TTFT + (tokens − 1) × gap · 500 tokens × 20 ms = about 10 s of writing: stream the tokens

3 My measurement setup

Item	Value
model	6 layers, 384 wide, 6 heads, 8,000-word vocabulary, **16.8M** parameters, **random weights**
machine	one CPU thread, float32, best of several tries
purpose	time the phases; not a quality test

4 Prefill, measured



Prompt	Prefill	Per token
64 tokens	22.6 ms	0.35 ms
256 tokens	62.8 ms	0.25 ms
1,024 tokens	266.6 ms	0.26 ms

Script: l06_decode_kv.py.

5 Prefill against decode

phase	tokens per second on the same small model (measured)
prefill	4,076 tokens/s
decode, with cache	182 tokens/s

a decode token costs about 19 prefill tokens

Prefill: 256 tokens in 62.8 ms = about **4,076 tokens/s**. Decode with a cache: one step 4.7 ms = about **182 tokens/s**. A decode token costs roughly **19** prefill tokens.

6 The KV cache, measured

setting	time to write 64 tokens after a 256-token prompt (measured)
with a KV cache	0.35 s (182 tokens/s)
without a cache	4.25 s (15 tokens/s)

12.1 times slower without the cache

64 new tokens after a 256-token prompt	Time	Tokens/s
with a KV cache	0.35 s	181.8
without a cache (re-run everything each step)	4.25 s	15.1

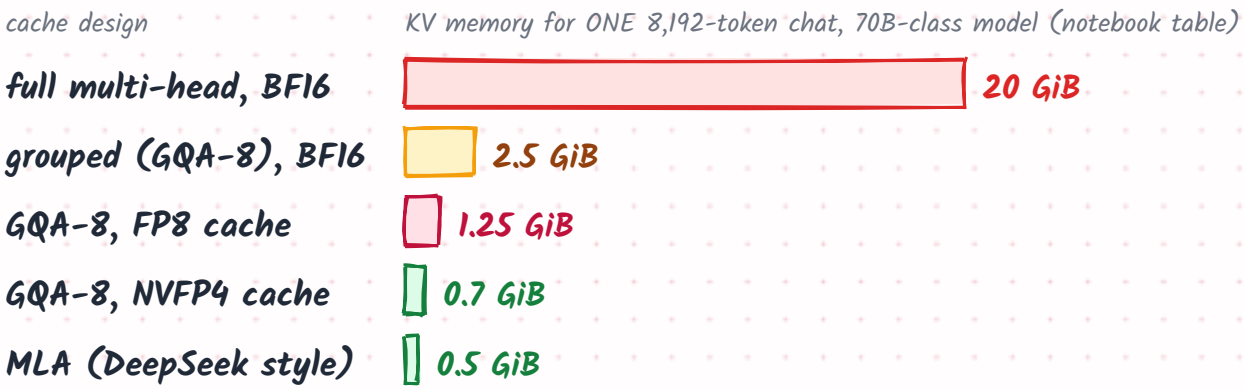
> **12.1x slower** without the cache, and the gap grows with longer answers.

7 The cache formula, checked

> **KV bytes per token = 2 × layers × KV heads × head size × bytes per number**

Check	Value
formula for my model	2 × 6 × 6 × 64 × 4 = 18,432 bytes per token
real tensors after a 256-token prompt	4,718,592 bytes ÷ 256 = **18,432**

8 Cache size for a 70B-class model



Notebook table (80 layers, 8 KV heads, head size 128). GQA-8 BF16 = 327,680 bytes ≈ 320 KiB per token; full multi-head is 8x bigger; MLA (DeepSeek style) about 69 KiB per token. Arithmetic, not measured here.

9 How many chats fit?

> 2 × 80 GB GPUs, 70B in FP8 (about 70 GB of weights) leave roughly 70 to 80 GB for cache. At 320 KiB per token that is about **30 concurrent 8K-token chats**. An FP8 cache doubles it: the first lever before buying GPUs.

10 Paging and prefix caching

- → **PagedAttention** (vLLM): small blocks on demand, like OS pages; waste under one block per chat
- → **Prefix caching**: requests that share a start reuse its keys and values, so their prefill is skipped. Put shared content first, byte-identical
- → Offload tiers (CPU memory, local NVMe, network storage) pay off for long prefixes and multi-turn chats

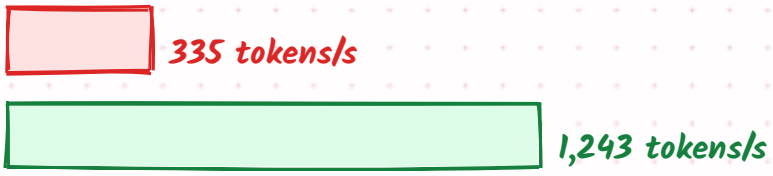
11 Static or continuous batching?

Policy	How it works
static	take 32 requests, run until the LONGEST finishes, then the next 32
continuous	after every step finished requests leave and waiting ones join at once

12 Batching, simulated

Setting	Value
requests	2,000 arriving together, mean 119 output tokens (exponential, capped at 600, assumed)
GPU	32 slots, 25 ms per step regardless of batch (assumed)

policy — throughput, 2,000 requests, 32 slots (simulation, assumed 25 ms step)



3.7 times the output from the same GPU

policy — average time until a request finishes (simulation)



short requests no longer wait for the longest

Policy	Total time	Tokens/s	Mean finish
static	710 s	335	366 s
continuous	192 s	1,243	95 s

Script: l06_batching_sim.py.

13 Chunked prefill

→ A long prefill can hog a whole step: every other user's reply freezes (a stall of seconds)

→ Fix: a per-step **token budget**: decodes first, then a slice of the long prompt (--max-num-batched-tokens in vLLM)

→ Result: smooth gaps, slightly later first token for the long prompt

14 Disaggregation



use only when the gain beats the cost of moving KV

Disaggregate when	Stay aggregated when
long prompts, strict smoothness targets, big MoE, many nodes	a few GPUs only, KV transfer costs more than the interference, a fixed prefill:decode ratio goes stale

15 Which engine?

Engine	Pick when (status Oct 2026, notebook)
vLLM	the default: widest model and hardware support, OpenAI-style API
SGLang	shared prefixes, agents, big MoE
TensorRT-LLM	all-NVIDIA, chasing the last few percent
Dynamo / llm-d	multi-node routing and prefill/decode split on top
HF TGI	maintenance mode: migrate to vLLM or SGLang
Ollama / llama.cpp	laptop and edge, fewer than ~20 users

16 A launch command (sketch)

```
vllm serve MODEL_NAME \
  --tensor-parallel-size 2 \
  --quantization fp8 \
  --kv-cache-dtype fp8 \
  --max-model-len 32768 \
  --gpu-memory-utilization 0.90 \
  --max-num-batched-tokens 8192
```

From the notebook; verify flags for your version.

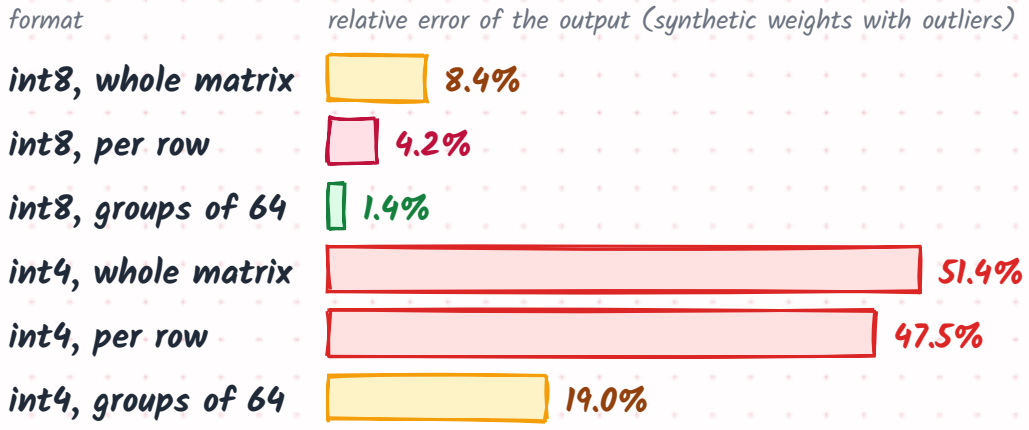
17 Quantisation

Decode time ∝ bytes read: 8-bit is about 2x faster than 16-bit when memory-bound, 4-bit about 4x, and the cache gets more room.

Format	Notes (notebook)
BF16	reference
FP8	near-lossless, the 2026 default on Hopper and Blackwell
INT4 (AWQ / GPTQ)	weights only; the maths stays 16-bit
NVFP4	true 4-bit maths, Blackwell only

18 Rounding weights, measured

Synthetic 1,024 x 1,024 weights, 8 outlier columns 20x larger (assumed shape). 200 random inputs. Error = |Wx − W_q x| ÷ |Wx|. Plain rounding only. Script: l06_quant_error.py.



19 Quantisation results

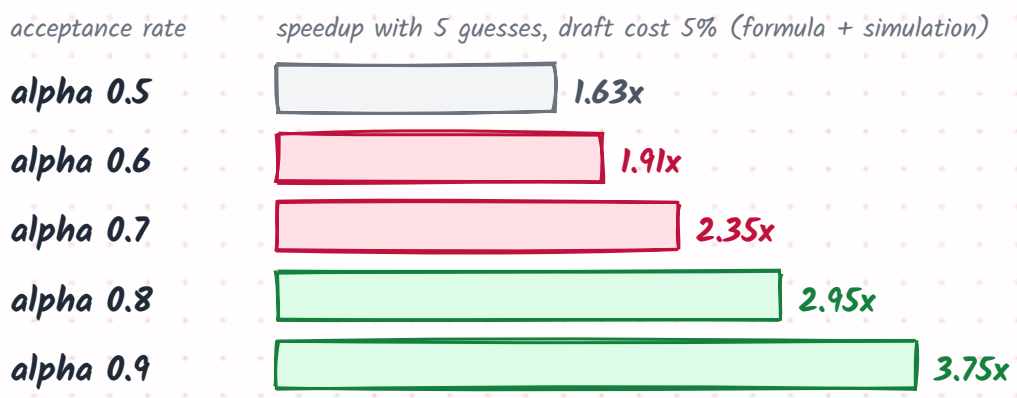
Format	Bytes per weight	Relative error
16-bit (float16 stand-in)	2.00	0.00021
int8, whole matrix	1.00	0.0844
int8, per row	1.00	0.0419
int8, groups of 64	1.06	**0.0141**
int4, whole matrix	0.50	**0.514**
int4, per row	0.50	0.475
int4, groups of 64	0.56	0.190

Not good enough
Plain 4-bit rounding broke the matrix. Real methods (AWQ, GPTQ) are cleverer and were not tested here. Smaller scale groups help every time.

20 Speculative decoding

A cheap draft guesses γ tokens; the big model checks all of them in **one pass**, keeps the accepted run plus one more token. The text is exactly the big model's.

tokens per pass = $(1 - \alpha^{\gamma+1}) \div (1 - \alpha)$ speedup ≈ tokens per pass ÷ $(1 + \gamma \times c)$



real-world: about 2 to 3x at low batch size

α	γ	Simulated	Formula	Speedup (c = 0.05)
0.55	5	2.16	2.16	1.73x
0.70	5	2.95	2.94	**2.35x**
0.80	5	3.69	3.69	2.95x
0.90	5	4.69	4.69	3.75x

SIMULATION, 200,000 passes. α, γ, c are assumed inputs. Script: l06_spec_decode.py.

21 Reality check, FlashAttention, and APIs

- ✓ Speculation in practice: about 2 to 3x on small models at small batch; it fades at large batch; one 8B test: 1.84x on code, 0.99x on chat; below $\alpha \approx 0.55$ it loses
- ✓ FlashAttention never stores the big score table; the engines switch it on for you
- ✓ Managed APIs: on-demand, batch (about 50% off), priority tiers, prompt caching (repeated input about 10%). Self-host only at high, steady use (lesson 11)

22 ShopNest Copilot: serving plan

Piece	Choice
engine	vLLM, 8B model
bytes	FP8 weights + FP8 cache
reuse	prefix caching: every request starts with the same store instructions
batching	continuous + chunked prefill
speculation	only if the measured acceptance rate is good

23 Words to keep

Word	Meaning
TTFT	time to first token: the wait before the first word
KV cache	stored keys and values so earlier words are not recomputed
Continuous batching	refill a slot the moment a request finishes
Quantisation	fewer bits per number: less memory, faster decode

24 Interview corner

Q. Why is decode slower than prefill?

A. Prefill is parallel and compute-bound; decode makes one token per step and is memory-bound. In my test a decode token cost about 19 prefill tokens.

Q. What does the KV cache do, and what does it cost?

A. It stores earlier keys and values so each step computes only the new word. My small model was 12x faster with it. The cost is memory: 2 × layers × KV heads × head size × bytes per token.

Q. How do you raise throughput?

A. Continuous batching (3.7x in my simulation), a quantised KV cache, prefix caching, chunked prefill.

Q. When does speculative decoding help?

A. When the draft is accepted often (alpha above about 0.55) and the batch is small. Measure on your own traffic.

25 Your turn

106_decode_kv.py, **106_batching_sim.py**, **106_quant_error.py**, **106_spec_decode.py**: change answer length, slots, group size.

⚡ Revise in 60 seconds

- → prefill parallel · decode waits on memory
- → KV cache: **12.1x** faster, 18,432 B per token (small model)
- → decode token ≈ **19** prefill tokens
- → cache for 70B at 8K tokens ≈ 2.5 GiB (GQA-8)
- → continuous batching: **3.7x** throughput (sim)
- → int8 groups of 64: **1.4%** error · plain int4: **51%**
- → speculation: 2.35x at α 0.7 (sim), about 2 to 3x in practice
- → size by goodput, not raw tokens/s

Next lesson → CAI-07
MLOps and LLMOps: how to ship all of this safely, again and again
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**